Restaurant Food Recognition for Calorie Estimation

Group - 29 Team Guided by Dr. Prof Angshuman Paul

Introduction

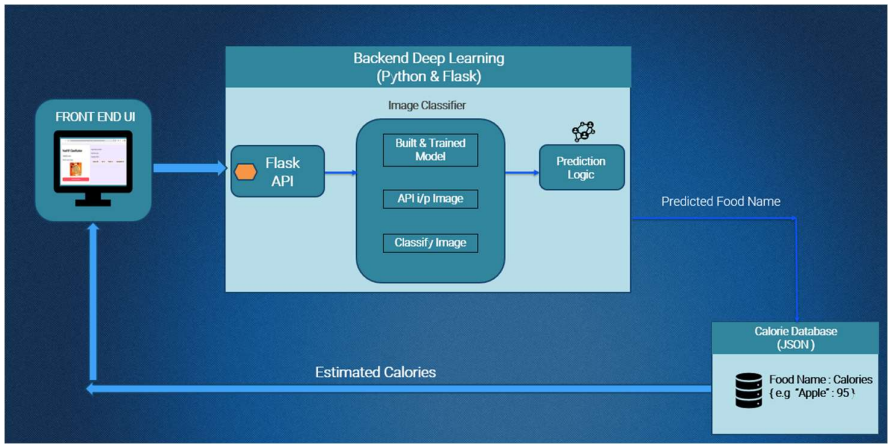
People are generally considered obese when their Body Mass Index (BMI) is over 30 kg/m2. High BMI is associated with the increased risk of diseases, such as heart disease. Unfortunately, more and more people will meet criteria for obesity. The main cause of obesity is the imbalance between the amount of food intake and energy consumed by the individuals. Obesity treatment requires the patients to eat healthy food and decrease the amount of daily calorie intake, which needs patients to calculate and record calories from foods every day.

Key Contributions of the Project

Name	Key Contributions
Rayipalli Prakash (M24AID031)	Coordinated project tasks, designed architecture, implemented backend logic, and integrated APIs
Ch V V S Suresh Datta (M24AID009)	Data preprocessing, performed model building and model testing.
Prathamesh Lingam (M24AID028)	Created responsive UI using HTML/CSS, improved usability, design consistency and documentation.
Pratik Deepak Kumbhar (M24AID024)	Prepared report, docs and visualized results.

System Architecture

This architecture outlines a system designed to identify food items from images and estimate their calorie content.



Frontend UI (User Interface)

This is the user-facing part of the application, likely implemented as a web or mobile interface.

- Allows users to upload a food image.
- Sends the uploaded image as input to the Backend Deep Learning service.

Backend Deep Learning (Flask API Framework)

This is the core intelligence of the system.

- Technologies: Python, using a web framework such as Flask or FastAPI for API endpoints, along with deep learning libraries like PyTorch.
- ResNet50 Based Food Classifier: A pre-trained ResNet50 model (or another suitable CNN) that has been fine-tuned on a food dataset.
- Prediction Logic: This component takes the image input, processes it, and uses the trained ResNet50 model to predict the class (name) of the food item.
- Food-101 data set has been used for training the base model.

Calorie Database (JSON)

A simple database storing food names and their corresponding calorie values.

- Each entry contains a food item name and its associated calorie count per gm.

Model Training and Development

A ResNet-based Convolutional Neural Network (CNN) serves as the backbone for food recognition. Residual skip connections are incorporated to mitigate vanishing gradients and enable the training of deeper networks. The final classification layer outputs 101 probabilities, each representing a distinct food category.

Once classification is complete, the predicted label is passed to a nutritional database to retrieve the estimated calorie values, forming a bridge between visual recognition and dietary analytics. The modular design ensures scalability, allowing new food categories or additional nutritional data to be seamlessly integrated without retraining the entire system.

Models explored

- ResNet18
- MobileNetV3_Large
- ResNet50
- Swin-Transformer

A part from ResNet50, Swin-Transformer consuming more processing time with less loss convergence. Remaining models were with low accuracy.

Models Architecture

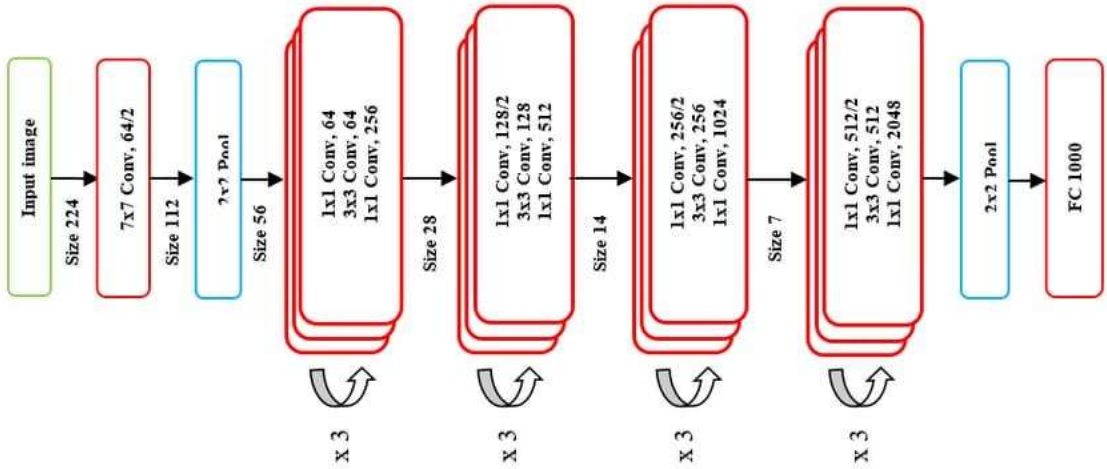
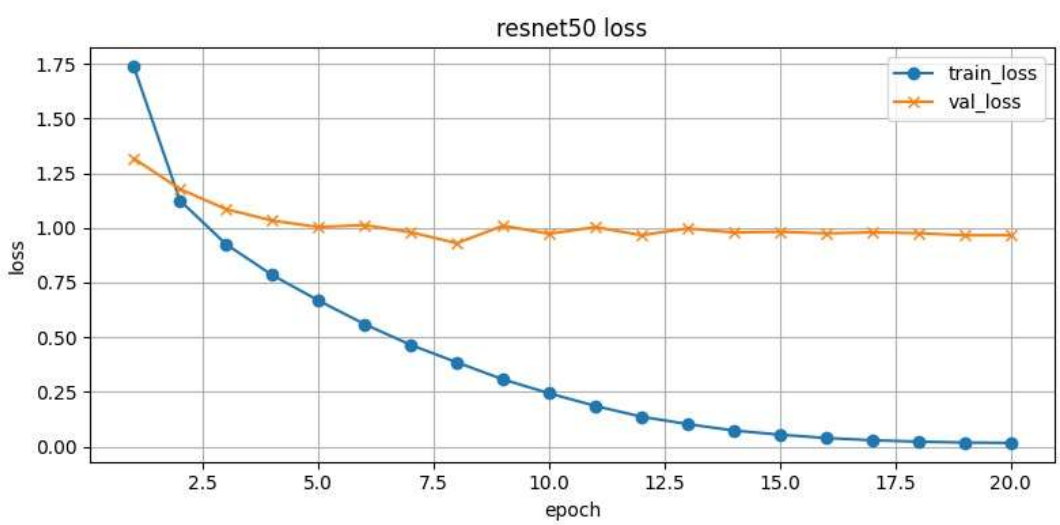


Figure 1 ResNet50 base architecture

Fine Tuned Parameters:

Parameter	Description
Model	ResNet50 (pretrained = True)
Final Layer Modification	Final fully connected layer replaced to match 101 output classes
Loss Function	CrossEntropyLoss
Optimizer	Adam (learning rate = 0.0002)
Learning Rate Scheduler	Cosine Annealing
Epochs	20 (for convergence)
Batch Size	64 (depending on GPU memory)
Input Image Size	224 × 224 pixels

Epoch vs loss Graph



Training Results

epoch	train_loss	val_loss	val_acc	Learningrate	epoch_time_sec	avg_batch_sec
1	1.737653	1.317772	0.651947	0.000199	214.0602	0.134282887
2	1.125578	1.177923	0.693399	0.000195	184.0386	0.136338659
3	0.925086	1.085593	0.719208	0.000189	200.2535	0.148477475
4	0.784321	1.033612	0.732871	0.000181	204.8306	0.151702739
5	0.668181	1.00393	0.74363	0.000171	204.8011	0.151776372
6	0.559907	1.012889	0.749175	0.000159	203.7853	0.151043904
7	0.465189	0.980204	0.758812	0.000145	204.8788	0.151917533
8	0.38531	0.930152	0.777426	0.000131	205.2435	0.151795483
9	0.307264	1.010241	0.764158	0.000116	205.003	0.151869447
10	0.243674	0.973358	0.77967	0.0001	205.1975	0.151876771
11	0.185943	1.003698	0.775974	8.44E-05	205.5509	0.15218182
12	0.136726	0.966775	0.787129	6.91E-05	204.8394	0.151701404
13	0.10292	0.997765	0.791155	5.46E-05	204.5183	0.151605745
14	0.073616	0.979059	0.796106	4.12E-05	204.7482	0.151605134
15	0.054678	0.98255	0.798416	2.93E-05	204.8597	0.15182393
16	0.039381	0.97514	0.808779	1.91E-05	205.078	0.151848827
17	0.029155	0.979955	0.806799	1.09E-05	204.8453	0.151764382
18	0.022759	0.975953	0.808449	4.89E-06	205.2328	0.15180342
19	0.018747	0.96619	0.810957	1.23E-06	205.3242	0.152072575
20	0.016909	0.96665	0.813135	0	204.6813	0.151701689

System Flow

1. The user interacts with the frontend UI and uploads an image of a food item.
2. The frontend UI sends this image (e.g., via an HTTP POST request) to the backend deep learning service.
3. The backend deep learning service receives the image.
4. The ResNet50-based food classifier processes the image and uses its trained model to predict the class (e.g., “Apple,” “Pizza,” “Salad”).
5. The backend deep learning service sends the predicted food class name back to the frontend UI.
6. The backend deep learning service uses the predicted food class name to query the calorie database (JSON).
7. The calorie database (JSON) looks up the provided food name and returns its estimated calorie value to the backend deep learning service.
8. The backend deep learning service sends the estimated calorie information back to the frontend UI.
9. The frontend UI displays both the predicted food name and its estimated calorie content to the user.